

GeCode CSP Pipeline - Arquitectura del Sistema

Diseno, Flujo de Datos y Componentes

Proyecto GeCode CSP Pipeline

2026

Contents

Vision General de la Arquitectura	3
Filosofia de Diseno	3
Principios de Diseno	3
Arquitectura de Alto Nivel	3
Componentes del Sistema	5
1. SyntaxChecker	5
2. JsonToGraph	5
3. FunctionChecker	5
4. FwdConsistency	5
5. BwdConsistency	6
6. TestGecodeBridge	6
Herramientas Auxiliares	6
CSPEval	6
ForwardChain	6
JsonSink / JsonSource	6
Flujo de Datos	7
Formato JSON de Entrada	7
Transformaciones por Etapa	7
Etapa 1: SyntaxChecker -> JSON Validado	7
Etapa 2: JsonToGraph -> Grafo con AST	8
Etapa 3: FunctionChecker -> Grafo Verificado	8
Etapa 4-5: Propagacion -> Dominios Reducidos	8
Etapa 6: TestGecodeBridge -> Soluciones	9
Arquitectura de Persistencia	10
SQLite como Almacenamiento Intermedio	10
Esquema de Base de Datos	10
Flujo con Persistencia	10
Ventajas de la Persistencia	10
Uso	10
Integracion de Lenguajes	11

Pascal + C/C++ + Gecode	11
1. Pascal (Free Pascal)	11
2. C (Biblioteca MiniMath)	11
3. C++ (Gecode Bridge)	11
Diagrama de Integracion	11
Sistema de Compilacion	13
Makefile Modular	13
Compilacion Monolitica	13
Modos de Linkeo	13
Diagrama de Compilacion	14

Vision General de la Arquitectura

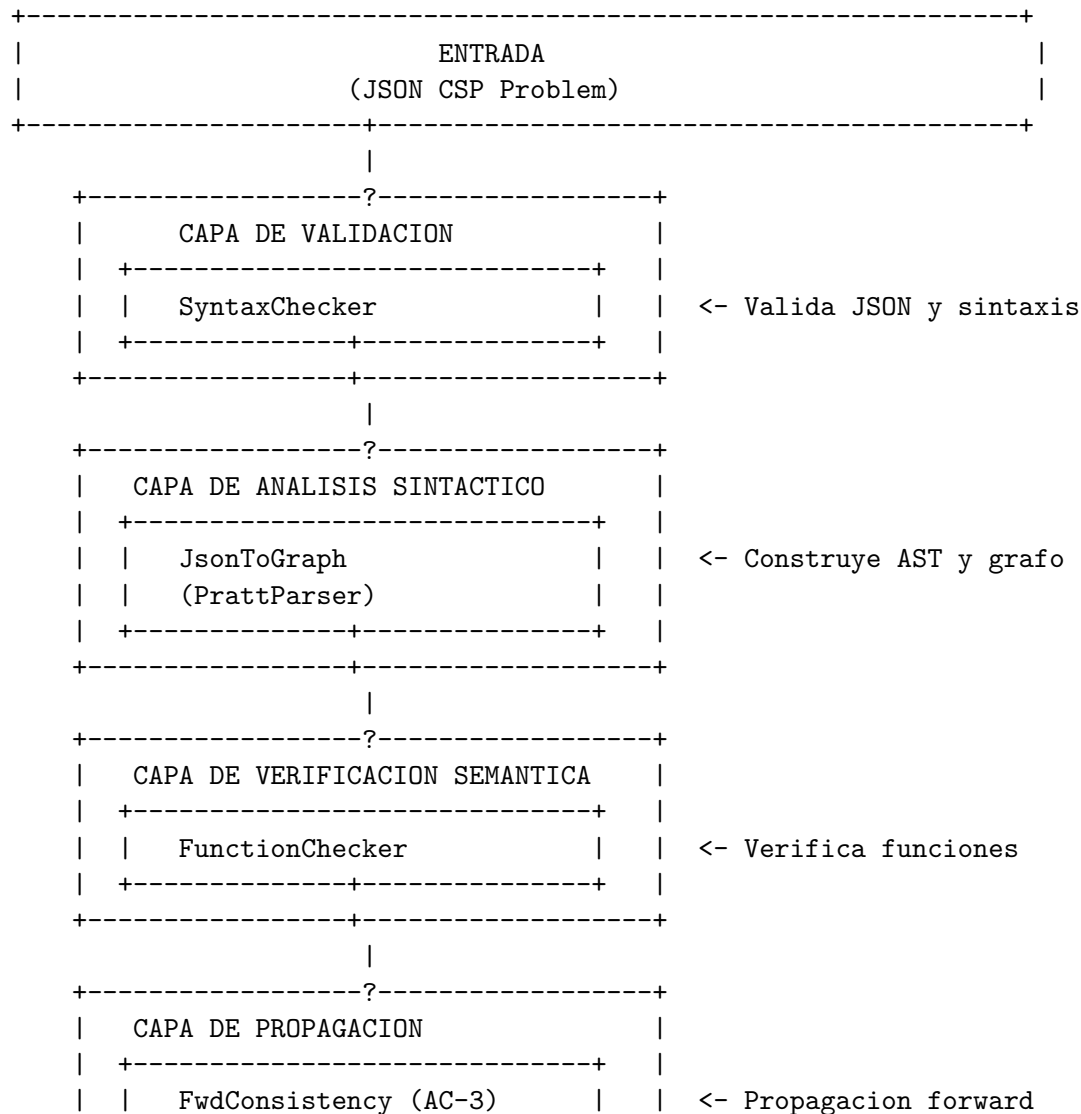
Filosofia de Diseno

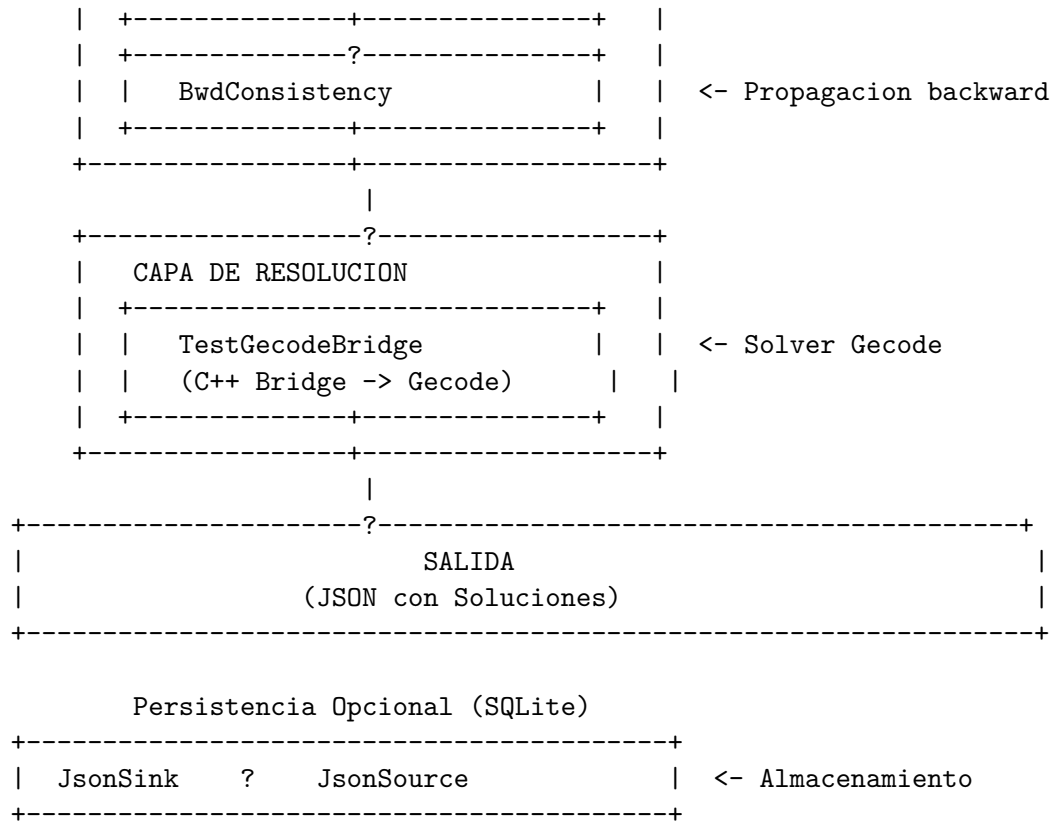
El GeCode CSP Pipeline sigue una arquitectura de **pipeline modular** donde cada componente tiene una responsabilidad unica y bien definida. Los datos fluyen secuencialmente a traves de etapas de transformacion y verificacion.

Principios de Diseno

1. **Separacion de Responsabilidades:** Cada herramienta hace una cosa y la hace bien
2. **Formato JSON:** Comunicacion entre componentes mediante JSON estructurado
3. **Composabilidad:** Las herramientas pueden usarse independientemente o en pipeline
4. **Persistencia Opcional:** SQLite permite almacenar/recuperar estados intermedios
5. **Compilacion Flexible:** Soporte para compilacion modular o monolitica

Arquitectura de Alto Nivel





Componentes del Sistema

1. SyntaxChecker

Proposito: Validar la sintaxis del archivo JSON de entrada

Entrada: Archivo JSON **Salida:** JSON validado o mensaje de error

Funciones:

- Verificar estructura JSON valida
- Validar campos obligatorios (precision, variables, expresiones)
- Verificar tipos de datos
- Reportar errores de sintaxis

Tecnologias: Pascal + MiniJSON

2. JsonToGraph

Proposito: Construir el AST y grafo de restricciones

Entrada: JSON validado **Salida:** Grafo JSON con AST de expresiones

Funciones:

- Parsear expresiones con PrattParser
- Construir arbol de sintaxis abstracta
- Generar grafo de dependencias
- Identificar nodos y aristas

Tecnologias: Pascal + PrattParser + ExpressionAST

3. FunctionChecker

Proposito: Verificar funciones definidas por el usuario

Entrada: Grafo JSON **Salida:** Grafo verificado

Funciones:

- Validar definiciones de funciones
- Verificar parametros y tipos
- Comprobar referencias validas
- Marcar funciones validas/invalidas

Tecnologias: Pascal + UCSPJson

4. FwdConsistency

Proposito: Propagacion de restricciones hacia adelante (AC-3)

Entrada: Grafo verificado **Salida:** Grafo con dominios reducidos

Funciones:

- Implementar algoritmo AC-3
- Reducir dominios de variables
- Propagar restricciones forward
- Detectar inconsistencias tempranas

Tecnologias: Pascal + MiniMath + AC-3 Algorithm

5. BwdConsistency

Proposito: Proyeccion inversa de restricciones

Entrada: Grafo con dominios reducidos **Salida:** Grafo con analisis de intervalos

Funciones:

- Analisis de intervalos
- Proyeccion inversa de restricciones
- Refinamiento de dominios
- Propagacion backward

Tecnologias: Pascal + MiniMath + Interval Analysis

6. TestGecodeBridge

Proposito: Resolucion CSP completa con Gecode

Entrada: Grafo optimizado **Salida:** JSON con soluciones

Funciones:

- Traducir CSP a modelo Gecode
- Configurar variables y restricciones
- Ejecutar busqueda
- Retornar soluciones encontradas

Tecnologias: Pascal + C++ Bridge + Gecode Solver

Herramientas Auxiliares

CSPEval

Evaluador de expresiones CSP con calculo de resultados

ForwardChain

Encadenamiento hacia adelante para inferencia logica

JsonSink / JsonSource

Persistencia y recuperacion desde SQLite

Flujo de Datos

Formato JSON de Entrada

```
{
  "precision": 2,
  "variables": [
    {
      "nombre": "x",
      "tipo": "integer",
      "domain": [1, 100],
      "value": 10
    },
    {
      "nombre": "y",
      "tipo": "float",
      "domain": [0.0, 50.0],
      "value": 15.5
    }
  ],
  "expresiones": [
    "x + y * 2",
    "x > 5"
  ],
  "funciones": [
    {
      "nombre": "miFuncion",
      "parametros": ["a", "b"],
      "salida": "a + b * 2"
    }
  ]
}
```

Transformaciones por Etapa

Etapas: SyntaxChecker -> JSON Validado

```
{
  "valido": true,
  "errores": [],
  "precision": 2,
  "variables": [...],
  "expresiones": [...]
}
```

Etapla 2: JsonToGraph -> Grafo con AST

```
{
  "precision": 2,
  "variables": [...],
  "expresiones": [
    {
      "texto": "x + y * 2",
      "ast": {
        "tipo": "BINOP",
        "op": "+",
        "izq": {"tipo": "IDENT", "nombre": "x"},
        "der": {
          "tipo": "BINOP",
          "op": "*",
          "izq": {"tipo": "IDENT", "nombre": "y"},
          "der": {"tipo": "LITERAL", "valor": 2}
        }
      }
    }
  ],
  "grafo": {
    "nodos": [...],
    "aristas": [...]
  }
}
```

Etapla 3: FunctionChecker -> Grafo Verificado

```
{
  "funciones_verificadas": true,
  "funciones": [
    {
      "nombre": "miFuncion",
      "valida": true,
      "mensaje": "OK"
    }
  ],
  ...
}
```

Etapla 4-5: Propagacion -> Dominios Reducidos

```
{
  "variables": [
    {
      "nombre": "x",

```



```

    "domain_original": [1, 100],
    "domain_reducido": [6, 100]  <- Despues de x > 5
  }
],
...
}

```

Etapa 6: TestGecodeBridge -> Soluciones

```

{
  "num_soluciones": 3,
  "soluciones": [
    {
      "x": 10,
      "y": 15.5
    },
    {
      "x": 11,
      "y": 14.25
    },
    {
      "x": 12,
      "y": 13.0
    }
  ],
  "estadisticas": {
    "tiempo_ms": 42,
    "nodos_explorados": 156
  }
}

```

Arquitectura de Persistencia

SQLite como Almacenamiento Intermedio

El sistema permite **persistir cada etapa del pipeline** en una base de datos SQLite mediante las herramientas **JsonSink** y **JsonSource**.

Esquema de Base de Datos

```
CREATE TABLE json_storage (  
    tag TEXT PRIMARY KEY,  
    timestamp INTEGER,  
    json_data TEXT  
);
```

Flujo con Persistencia

Input JSON

?

```
SyntaxChecker -> JsonSink(db, "problema1_syntax")  
                  ?
```

```
                  JsonSource(db, "problema1_syntax")
```

?

```
JsonToGraph -> JsonSink(db, "problema1_graph")
```

?

```
                  JsonSource(db, "problema1_graph")
```

?

```
FunctionChecker -> JsonSink(db, "problema1_checked")
```

?

...

Ventajas de la Persistencia

1. **Debugging:** Inspeccionar estado en cualquier etapa
2. **Reproducibilidad:** Re-ejecutar desde cualquier punto
3. **Analisis:** Comparar resultados de multiples ejecuciones
4. **Cacheo:** Evitar re-computar etapas costosas

Uso

```
# Almacenar JSON  
echo '{"data": "value"}' | ./bin/JsonSink db.sqlite mi_tag  
  
# Recuperar JSON  
./bin/JsonSource db.sqlite mi_tag  
  
# Pipeline completo con persistencia  
./scripts/pipeline.sh --db runs.db --tag test_1 input.json
```

Integracion de Lenguajes

Pascal + C/C++ + Gecode

El proyecto integra **tres lenguajes** de forma eficiente:

1. Pascal (Free Pascal)

Usado para: - Herramientas del pipeline - Parsing y validacion - Logica de negocio - Interfaz de linea de comandos

Ventajas: - Compilacion rapida - Ejecutables eficientes - Sintaxis clara y legible - Excelente para procesamiento de texto

2. C (Biblioteca MiniMath)

Usado para: - Funciones matematicas (sin, cos, exp, ln) - Operaciones de intervalos - Calculos de precision

Integracion:

```
{ $LINKLIB minimath }
{ $L minimath_trig.o }
{ $L minimath_exp.o }
{ $L minimath_interval.o }

function c_sin(x: cdouble): cdouble; cdecl; external;
function c_cos(x: cdouble): cdouble; cdecl; external;
```

3. C++ (Gecode Bridge)

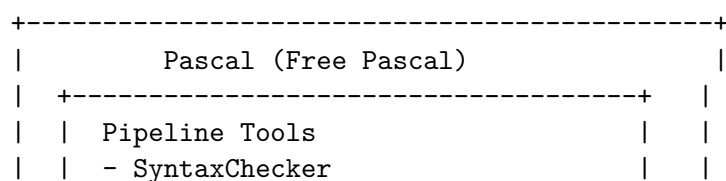
Usado para: - Interfaz a Gecode - Resolucion CSP - Busqueda y optimizacion

Integracion Pascal -> C++:

```
// UGecodeBridge.pas
function csp_create(vars: PVariable; n: cint): pointer; cdecl; external;
function csp_solve(model: pointer; sol: PSolution): cint; cdecl; external;

// gecode_bridge.cpp
extern "C" {
    void* csp_create(Variable* vars, int n) { ... }
    int csp_solve(void* model, Solution* sol) { ... }
}
```

Diagrama de Integracion



- JsonToGraph	
- FwdConsistency	
+-----+	
{\$LINKLIB}	
?	
+-----+	
MiniMath (C)	
- minimath_trig.o	
- minimath_exp.o	
- minimath_interval.o	
+-----+	
+-----+	
+-----+	
Pascal + C++ Bridge	
+-----+	
TestGecodeBridge.pas	
(Programa Pascal)	
+-----+	
external cdecl	
?	
+-----+	
UGecodeBridge.pas	
(Interface Unit)	
+-----+	
FFI	
?	
+-----+	
gecode_bridge.cpp	
(C API Wrapper)	
+-----+	
?	
+-----+	
Gecode (C++ Library)	
- IntVarArray	
- Constraints	
- Search	
+-----+	
+-----+	

Sistema de Compilacion

Makefile Modular

El Makefile permite compilar componentes de forma **selectiva**:

```
all: pipeline gecode

pipeline: minimath json_tools propagation_tools persistence_tools

gecode: gecode_bridge gecode_tools

minimath:
    gcc -c src/minimath_*.c

json_tools:
    fpc src/JsonToGraph.pas
    fpc src/SyntaxChecker.pas

gecode_bridge:
    g++ -c src/gecode_bridge.cpp -I$(GECODE_HOME)/include

gecode_tools:
    ./scripts/build_monolithic.sh src/TestGecodeBridge.pas
```

Compilacion Monolitica

El script build_monolithic.sh genera **ejecutables standalone**:

```
#!/bin/bash
# 1. Compilar bridge C++
g++ -c gecode_bridge.cpp

# 2. Compilar Pascal con linkeo explicito
fpc TestGecodeBridge.pas \
    -FU obj/ \
    -k'-L/path/to/gecode/lib' \
    -k'-lgecodekernel' \
    -k'-lgecodesupport' \
    -k gecode_bridge.o

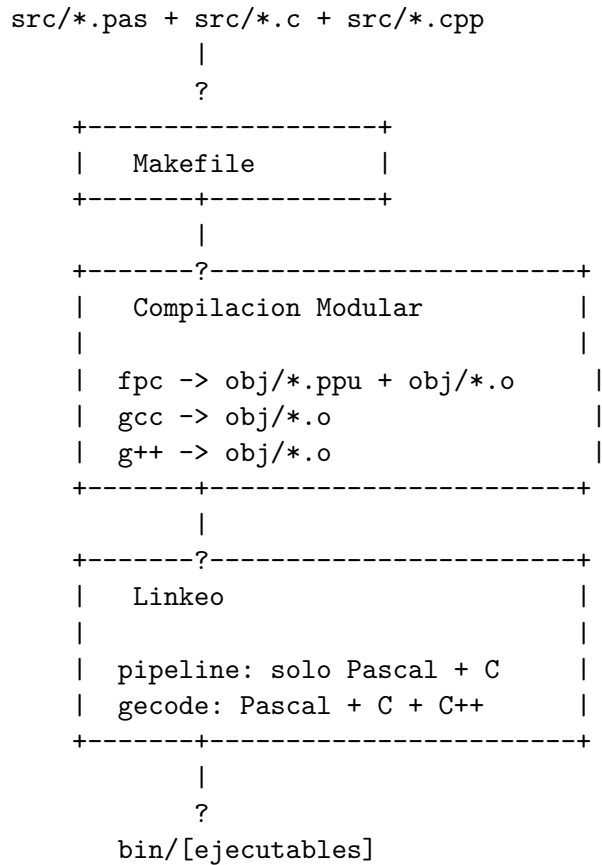
# Resultado: ejecutable standalone de ~8-12 MB
```

Modos de Linkeo

1. **Estatico**: Si existe \$GECODE_HOME/lib/*.a
 - Linkea libgecode*.a estaticamente
 - No requiere libgecode*.so en runtime
 - Ejecutable mas grande (~12 MB)
2. **Dinamico**: Si solo hay /usr/lib/libgecode*.so

- Linkea dinamicamente
- Requiere libgecode*.so instaladas
- Ejecutable mas pequeno (~500 KB)

Diagrama de Compilacion



Proximo: Leer `03_componentes_pipeline.md` para detalles tecnicos de cada componente.